Lab 3 - Clean the NYC Taxi Data

INF4058 Data Engineering

Name	**Yousef Abdellatef Yousef Hamoda**
Student ID	**692300249**
Date	19 October 2026
Repo link	https://github.com/yousefgm1/assignment-1-data-engneering

2. The flow, in my words



Source

The data is the NYC Yellow Taxi trip file for January 2021 (*yellow_tripdata_2021-01.csv.gz*, 1,369,765 rows), a public file that I treat as read-only.

Extract

ingest_data.py reads the compressed CSV in chunks (for example 100,000 rows at a time) instead of loading the whole file, so memory use stays small and constant.

Transform

Columns get explicit types: the pickup and dropoff columns are parsed as datetimes, and count-like columns that contain nulls use the nullable **Int64** type. Plain int64 cannot hold nulls, and pandas would silently turn the column into float otherwise.

Load

Each chunk is written to the table *yellow_taxi_trips* in PostgreSQL. On the **first chunk only** the table is created with *if_exists='replace'* (clean start, correct schema); every later chunk uses *append*. Replacing on every chunk would delete the previous chunk each time.

Serve

Once the data is in Postgres it can be queried with SQL (for example *SELECT COUNT(*)*) and used by analysts or dashboards. In this lab we stop before this stage and only clean.

What each file does

File	Role
docker-compose.yaml	Starts the PostgreSQL container (service *pgdatabase*) and the other services together, creates their shared network, and sets credentials and ports.
Dockerfile	Packages ingest_data.py with Python and its dependencies into an image, so the ingestion runs the same way on any machine.
ingest_data.py	The ETL script: extract in chunks, transform types, load into Postgres.

3. What we ran in class

One sentence per step, followed by the screenshot taken in class (command + output + terminal prompt).

a) docker compose up - starts PostgreSQL (and pgAdmin) in containers so the database is available.

Screenshot 3.1

[SCREENSHOT HERE]
docker compose up : command + output + prompt

b) Run ingest, then COUNT - ingest_data.py loaded the file in chunks, and *SELECT COUNT(*)* returned 1,369,765 rows, matching the source file.

Screenshot 3.2

[SCREENSHOT HERE]
ingest run + COUNT = 1,369,765

c) docker build - turned ingest_data.py and its dependencies into a Docker image.

Screenshot 3.3

[SCREENSHOT HERE]
docker build : command + output + prompt

d) docker run into yellow_taxi_trips - the container ran the same script, connected to the database as *pgdatabase*, and loaded the data into the table yellow_taxi_trips.

Screenshot 3.4

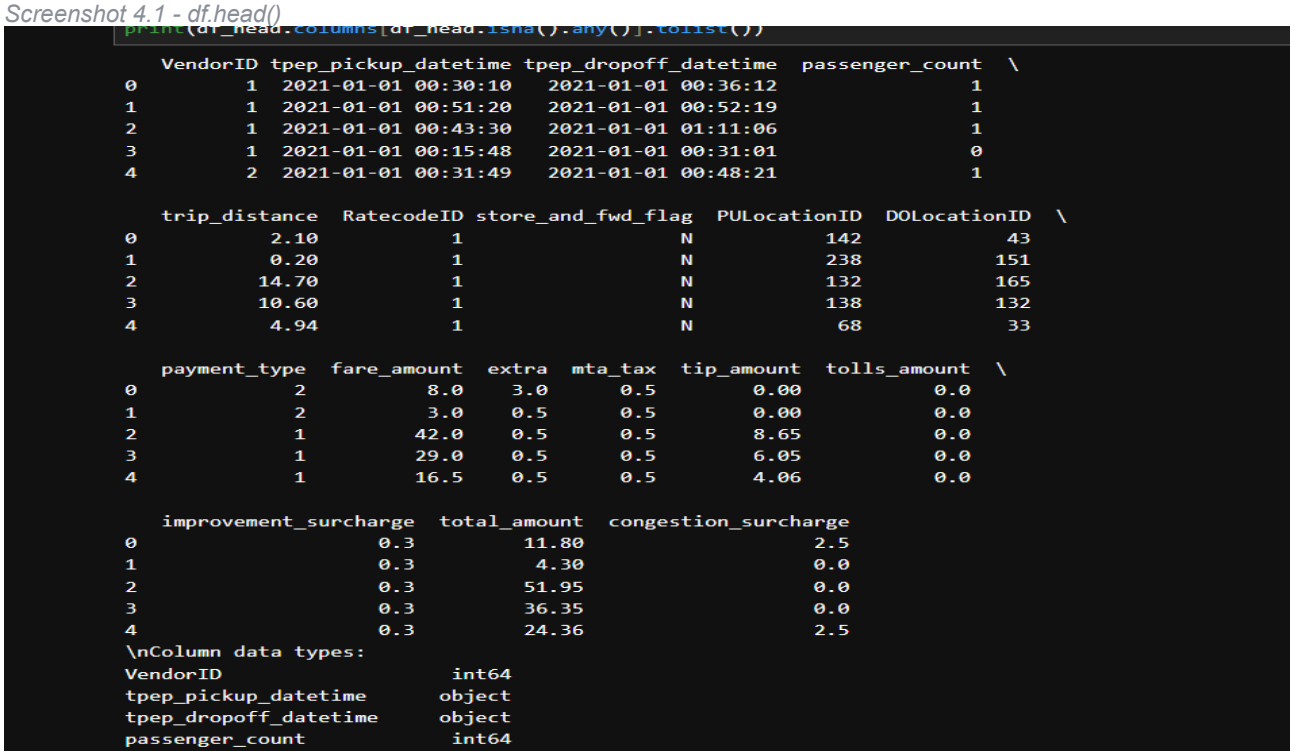
[SCREENSHOT HERE]
docker run : command + output + prompt

4. What I found in the data

All profiling was done on the original data, read in chunks of 100,000 rows, without modifying anything. Every table below is produced with *print()*.

4.1 First look (100 rows)

Screenshot 4.1 - df.head()



```
print(df_head.columns[df_head.isna().any()].tolist())

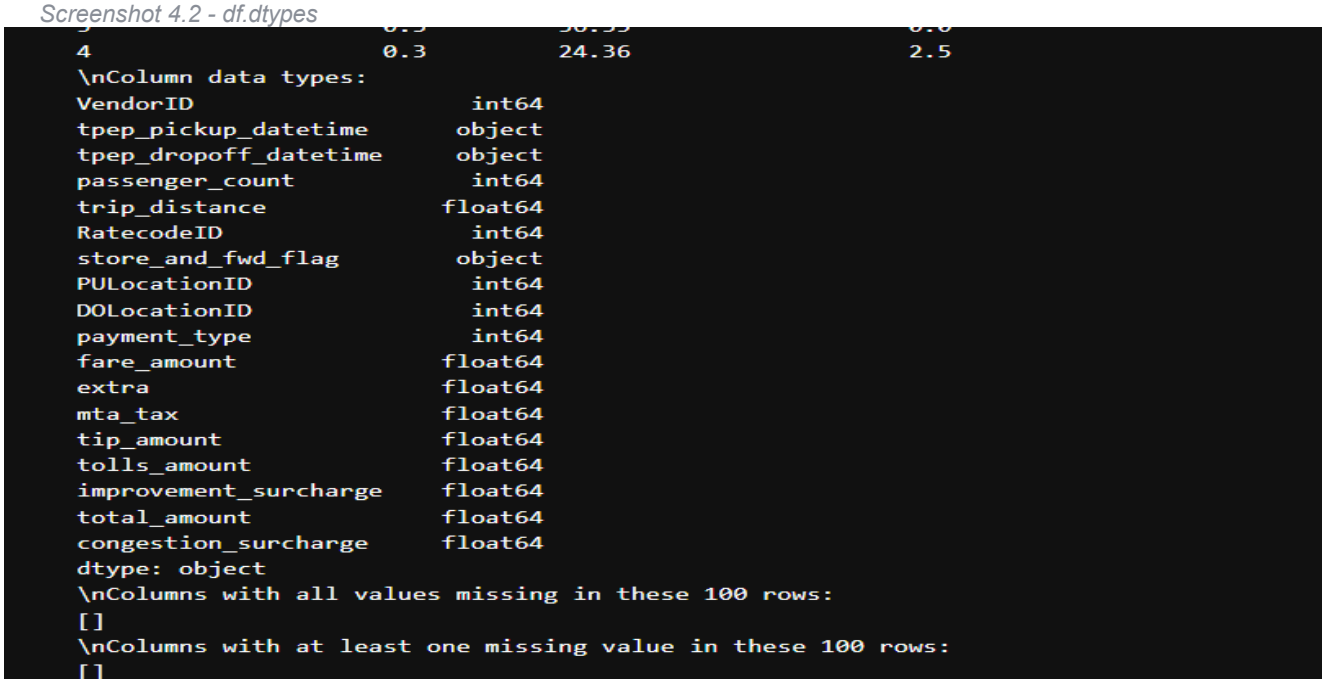
   VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  \
0         1  2021-01-01 00:30:10   2021-01-01 00:36:12                1
1         1  2021-01-01 00:51:20   2021-01-01 00:52:19                1
2         1  2021-01-01 00:43:30   2021-01-01 01:11:06                1
3         1  2021-01-01 00:15:48   2021-01-01 00:31:01                0
4         2  2021-01-01 00:31:49   2021-01-01 00:48:21                1

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID  \
0           2.10           1                  N           142            43
1           0.20           1                  N           238           151
2          14.70           1                  N           132           165
3          10.60           1                  N           138           132
4           4.94           1                  N            68            33

   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
0             2          8.0    3.0      0.5        0.00           0.0
1             2          3.0    0.5      0.5        0.00           0.0
2             1         42.0    0.5      0.5        8.65           0.0
3             1         29.0    0.5      0.5        6.05           0.0
4             1         16.5    0.5      0.5        4.06           0.0

   improvement_surcharge  total_amount  congestion_surcharge
0                    0.3         11.80                   2.5
1                    0.3          4.30                   0.0
2                    0.3         51.95                   0.0
3                    0.3         36.35                   0.0
4                    0.3         24.36                   2.5
\nColumn data types:
VendorID                 int64
tpep_pickup_datetime    object
tpep_dropoff_datetime   object
passenger_count          int64
```

Screenshot 4.2 - df.dtypes



```
4                    0.3         24.36                   2.5
\nColumn data types:
VendorID                 int64
tpep_pickup_datetime    object
tpep_dropoff_datetime   object
passenger_count          int64
trip_distance          float64
RatecodeID               int64
store_and_fwd_flag      object
PULocationID             int64
DOLocationID             int64
payment_type             int64
fare_amount            float64
extra                  float64
mta_tax                float64
tip_amount             float64
tolls_amount           float64
improvement_surcharge  float64
total_amount           float64
congestion_surcharge   float64
dtype: object
\nColumns with all values missing in these 100 rows:
[]
\nColumns with at least one missing value in these 100 rows:
[]
```

4.2 Profile of the whole file

Screenshot 4.3 - profiling table

Total rows profiled: 1369765

	dtype	null_count	null_percent	min	max
VendorID	float64, int64	98352	7.18	1	2
tpep_pickup_datetime	object	0	0.00	2008-12-31 23:05:14	2021-02-22 16:52:16
tpep_dropoff_datetime	object	0	0.00	2008-12-31 23:07:22	2021-02-22 16:56:15
passenger_count	float64, int64	98352	7.18	0	8
trip_distance	float64	0	0.00	0.0	263163.28
RatecodeID	float64, int64	98352	7.18	1	99
store_and_fwd_flag	float64, object	98352	7.18	N	Y
PULocationID	int64	0	0.00	1	265
DOLocationID	int64	0	0.00	1	265
payment_type	float64, int64	98352	7.18	1	4
fare_amount	float64	0	0.00	-490.0	6960.5
extra	float64	0	0.00	-5.5	8.25
mta_tax	float64	0	0.00	-0.5	0.5
tip_amount	float64	0	0.00	-100.0	1140.44
tolls_amount	float64	0	0.00	-31.12	811.75
improvement_surcharge	float64	0	0.00	-0.3	0.3
total_amount	float64	0	0.00	-492.8	7661.28
congestion_surcharge	float64	0	0.00	-2.5	3.0

4.3 The problems, with counts

Screenshot 4.4

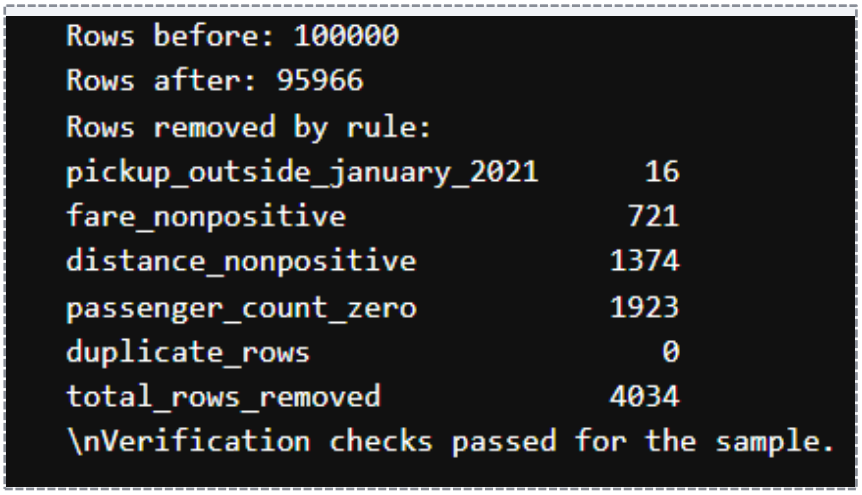
problem	count
pickup_outside_january_2021	24
fare_nonpositive	7411
distance_nonpositive	19952
passenger_count_zero	26726
duplicate_rows_after_first_occurrence	0

5. How I cleaned it

The function *clean(df)* returns the cleaned dataframe **and** a dictionary {rule: rows_removed}. It works on a copy, so the original dataframe is never changed and no row is dropped silently. Below is my draft decision table - edit it to match the choices you actually made.

Problem	Decision	Why / who owns the rule
Pickup outside Jan 2021	**Drop**	Row does not belong to this monthly file; timestamps are likely wrong. Owner: data engineering (ingestion contract).
Negative fare_amount	**Drop**	A fare cannot be negative in a trip record; may be refunds or errors. Owner: finance / business.
Zero fare_amount	**Drop**	A trip with no fare is not a normal paid trip. Owner: finance / business.
Negative trip_distance	**Drop**	Physically impossible. Owner: data engineering.
Zero trip_distance	**Flag**	May be a cancelled trip or same-place pickup; keep it but mark it so analysts decide. Owner: analytics team.
passenger_count = 0	**Flag**	Field entered by the driver, so 0 is often a missing value, not a bad trip. Owner: analytics team.
Exact duplicate rows	**Drop** (keep first)	Same trip counted twice distorts totals. Owner: data engineering.

Screenshot 5.2 - rows before and rows after



Check: rows before - sum of rows removed per rule = rows after 100000- 4034= 95966

6. Answers

Q1: Why does the container reach the DB as *pgdatabase*, not *localhost*?

Inside a container, *localhost* means the container itself, not my computer and not the database container. Docker Compose puts the services on a shared network with built-in DNS, where each service name is a hostname. The Postgres service is called *pgdatabase*, so the ingestion container finds it by that name. *localhost* only works from my own machine, through the published port.

Q2: Why chunks, and what was wrong with *replace* for many months?

Chunks: the file has 1.37 million rows (and other months are similar or larger). Loading it all at once uses a lot of memory and can crash the machine; chunks keep memory small and constant. Replace: *if_exists='replace'* drops and recreates the table. If it runs on every chunk, or every month, each write deletes the previous data and only the last piece remains. The fix is to use replace on the **first chunk only** and *append* afterwards.

Q3: How did I verify my results?

(Write this one yourself, based on what you actually did. Ideas: compare rows before - removed = rows after; re-run the profile on the cleaned dataframe and check the min/max values and that bad dates are gone; confirm duplicates = 0; confirm the original dataframe is unchanged; compare the total with the 1,369,765 from class.)

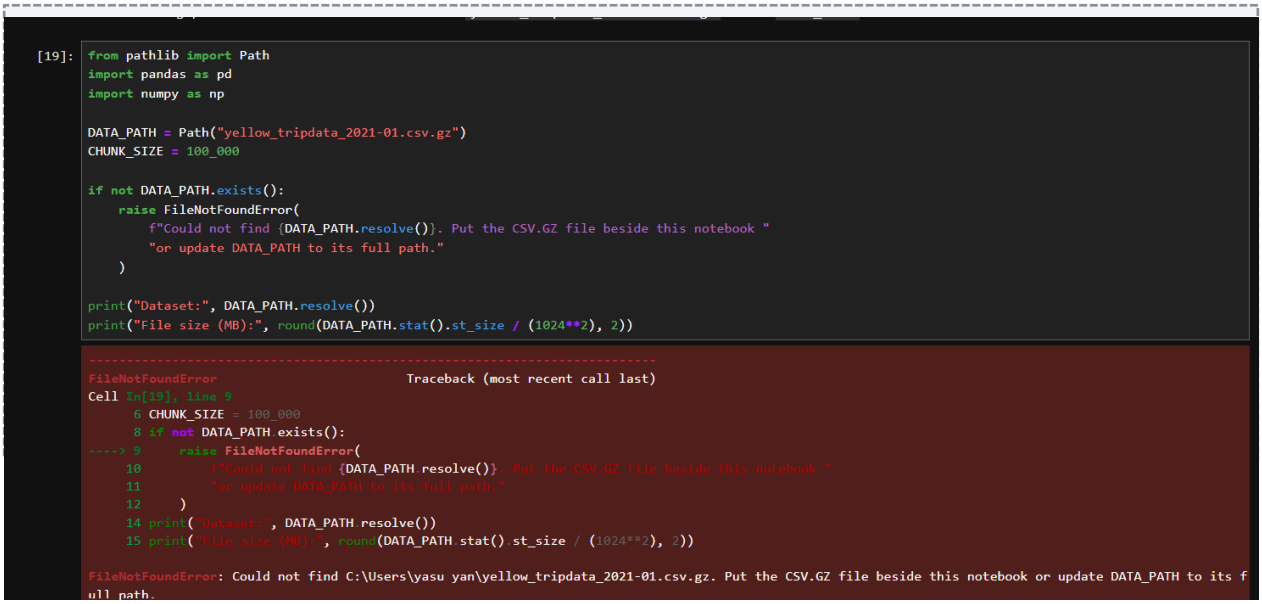
7. What broke

Error: can't find the file

Cause:

Fix: I put the file's path

Screenshot 7.1 - the error



```
[19]: from pathlib import Path
      import pandas as pd
      import numpy as np

      DATA_PATH = Path("yellow_tripdata_2021-01.csv.gz")
      CHUNK_SIZE = 100_000

      if not DATA_PATH.exists():
          raise FileNotFoundError(
              f"Could not find {DATA_PATH.resolve()}. Put the CSV.GZ file beside this notebook "
              "or update DATA_PATH to its full path."
          )

      print("Dataset:", DATA_PATH.resolve())
      print("File size (MB):", round(DATA_PATH.stat().st_size / (1024**2), 2))
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[19], line 9
      6 CHUNK_SIZE = 100_000
      8 if not DATA_PATH.exists():
----> 9     raise FileNotFoundError(
     10         f"Could not find {DATA_PATH.resolve()}. Put the CSV.GZ file beside this notebook "
     11         "or update DATA_PATH to its full path."
     12     )
     14 print("Dataset:", DATA_PATH.resolve())
     15 print("File size (MB):", round(DATA_PATH.stat().st_size / (1024**2), 2))

FileNotFoundError: Could not find C:\Users\yasu yan\yellow_tripdata_2021-01.csv.gz. Put the CSV.GZ file beside this notebook or update DATA_PATH to its f
ull path.
```

```
from pathlib import Path
import pandas as pd
import numpy as np

DATA_PATH = Path(r"E:\courses\7 دراسة\data engneering and cloud computing\assingment\yellow_tripdata_2021-01.csv.gz")
CHUNK_SIZE = 100_000

if not DATA_PATH.exists():
    raise FileNotFoundError(
        f"Could not find {DATA_PATH.resolve()}. Put the CSV.GZ file beside this notebook "
        "or update DATA_PATH to its full path."
    )

print("Dataset:", DATA_PATH.resolve())
print("File size (MB):", round(DATA_PATH.stat().st_size / (1024**2), 2))
```